You learned how to send emails in Chapter 13, *Task H: Sending Emails and Processing Payments Efficiently*, but when dealing with rich text and the need to send a plain-text email, we have to strip out the rich text. So, let's set up the mailer to respond to the user, and when we create the plain-text template, we'll see how to strip out the rich text. We'll start this off by creating the mailer using the Rails generator:

```
> bin/rails generate mailer support_request respond
      create  app/mailers/support_request_mailer.rb
      invoke  erb
      create    app/views/support_request_mailer
      create    app/views/support_request_mailer/respond.text.erb
      create    app/views/support_request_mailer/respond.html.erb
      invoke  test_unit
      create    test/mailers/support_request_mailer_test.rb
      create    test/mailers/previews/support_request_mailer_preview.rb
```

Our mailer will look similar to the mailers we've created in the past. This is what your **app/mailers/support_request_mailer.rb** should look like:

rails80/depot_tc/app/mailers/support_request_mailer.rb

```
class SupportRequestMailer < ApplicationMailer
  # Subject can be set in your I18n file at config/locales/en.yml
  # with the following lookup:
  #
  #   en.support_request_mailer.respond.subject
  #
» default from: "support@example.com"
»
» def respond(support_request)
»   @support_request = support_request
»   mail to: @support_request.email, subject: "Re: #{@support_request.
subject}"
» end
end
```

For the views, we'll show our response and quote the user's original email. As we saw in our web view, Rails will handle rendering the rich text for us,